

COURS DE CONCEPTION DES SYSTÈMES D'INFORMATION

Filière Informatique — Année Académique 2025-2026

PLATEFORME MARKETPLACE B2B FULLSTACK

RAPPORT DE PROJET : CONCEPTION, RÉALISATION ET DÉPLOIEMENT

Ce rapport présente la conception et la réalisation d'une plateforme web d'annonces interentreprises (B2B) fullstack, réalisée dans le cadre du cours de Conception des Systèmes d'Information. Il couvre l'analyse fonctionnelle (cas d'utilisation), la modélisation UML (diagrammes de classes et de séquences), le guide d'utilisation, ainsi que l'architecture de déploiement adoptée.

Réalisé par : [Prénom NOM 1] — [Prénom NOM 2]

Encadrant(e) : [Titre Prénom NOM du Professeur]

Technologies : Spring Boot (Java 17) | React Vite | PostgreSQL Neon | Docker

Déploiement : Backend AWS EC2 • Frontend Vercel • Base de données Neon

Date de soumission : Juin 2026

TABLE DES MATIÈRES

Ce sommaire présente la structure globale du document technique. Les sections et numéros de page correspondent à l'organisation séquentielle de ce rapport de projet.

INTRODUCTION ET CONTEXTE DU PROJET	3
1. Contexte du Projet et Cours de Conception SI	3
2. Problématique et Objectifs	3
3. Présentation Générale de l'Application Réalisée	4
CAHIER DES CHARGES ET SPÉCIFICATIONS FONCTIONNELLES	5
1. Acteurs du Système et Droits d'Accès	5
2. Spécifications Fonctionnelles par Cas d'Utilisation	5
3. Diagramme de Cas d'Utilisation Global du Système	6
ARCHITECTURE SYSTÈME ET CHOIX TECHNOLOGIQUES	7
1. Architecture Globale et Modèle Multi-tiers	7
2. Backend : Spring Boot (Java 17)	7
3. Frontend : React avec Vite	8
4. Base de Données : PostgreSQL Neon	8
5. Containerisation avec Docker	9
CONCEPTION DU SYSTÈME & MODÉLISATION UML	10
1. Diagramme de Classes Métier (Domain Model)	10
2. Diagramme de Classes Technique (Architecture en Couches)	11
3. Dictionnaire de Données et Structure des Tables SQL	12
MODÈLE DE SÉCURITÉ ET AUTHENTIFICATION JWT	13
1. Principes de la Sécurité Stateless avec JWT	13
2. Filtre de Sécurité dans Spring Boot	13
3. Gestion de la Politique CORS	14
CONCEPTION DÉTAILLÉE DES FLUX ET SÉQUENCES	15
1. Séquence : Publication d'un Produit par le Vendeur	15
2. Séquence : Contact Vendeur via WhatsApp	16
3. Séquence : Partage d'une Annonce avec un Ami	17
4. Implémentation du Partage (Web Share API)	17
DEVOPS ET WORKFLOW DE DÉPLOIEMENT AVEC DOCKER	19
1. Containerisation du Backend avec Dockerfile	19
2. Orchestration Locale avec Docker Compose	19
ARCHITECTURE DE DÉPLOIEMENT CLOUD	21
1. Vue d'Ensemble de l'Architecture Déployée	21
2. Hébergement Backend sur AWS EC2	21
3. Hébergement Frontend sur Vercel	22
4. Communication entre les Composants	22
GUIDE D'UTILISATION DE LA PLATEFORME B2B	23
1. Inscription, Connexion et Gestion du Profil	23
2. Recherche, Filtres par Catégorie et Favoris	23

3. Espace Vendeur : Publier, Modifier, Supprimer une Annonce	24
4. Contact Vendeur et Partage d'Annonces	24
VALIDATION ET TESTS DE L'APPLICATION	26
1. Tests Fonctionnels des Scénarios Clés	26
2. Résultats et Points d'Amélioration	26
CONCLUSION ET PERSPECTIVES	27

INTRODUCTION ET CONTEXTE DU PROJET

1. Contexte du Projet et Cours de Conception SI

Ce projet a été réalisé dans le cadre du cours de Conception des Systèmes d'Information. L'objectif est de concevoir et de développer une application web fullstack de type marketplace B2B (Business-to-Business), c'est-à-dire une plateforme d'annonces en ligne dédiée aux échanges entre professionnels. Le système permet à des vendeurs de publier des annonces de produits avec photos, et aux acheteurs de les consulter, de les mettre en favoris, de contacter directement le vendeur via WhatsApp, et de partager une annonce avec un tiers. L'application est construite avec une architecture multicouche (frontend, backend, base de données) et est déployée sur une infrastructure cloud distribuée.

2. Problématique et Objectifs

La problématique de ce projet est la suivante : comment concevoir et mettre en œuvre une plateforme web B2B permettant à des professionnels de publier, consulter et partager des annonces, avec une architecture sécurisée, modulaire et déployable sur le cloud ? Les objectifs fixés pour le projet sont :

- Sécurité des accès : Seuls les utilisateurs authentifiés peuvent publier ou modifier des annonces. La consultation du catalogue reste accessible à tous les visiteurs.
- Communication directe : Intégrer un mécanisme de contact via WhatsApp sans passer par un système de messagerie interne.
- Partage d'annonces : Permettre à un utilisateur de partager un lien d'annonce via l'API Web Share du navigateur ou par copie dans le presse-papiers.
- Déploiement cloud : L'application est hébergée sur une infrastructure cloud réelle (backend sur AWS EC2, frontend sur Vercel, base de données sur Neon DB).

3. Présentation Générale de l'Application Réalisée

L'application est un portail web fullstack organisé en deux espaces. L'espace public est accessible à tout visiteur sans authentification : il permet de consulter le catalogue de produits, d'effectuer des recherches par mots-clés, de filtrer par catégories, et de lire les fiches détaillées des annonces. L'espace privé, accessible aux utilisateurs connectés, donne accès au tableau de bord vendeur (publication, modification et suppression d'annonces), à la liste personnelle de favoris, ainsi qu'aux fonctionnalités de contact et de partage d'annonces. L'interface est construite en React (Vite). Le backend Spring Boot (Java 17) expose une API REST consommée par le frontend. La base de données PostgreSQL hébergée sur Neon stocke les utilisateurs, les produits et les associations de favoris.

[PÉRIMÈTRE DU PROJET] La plateforme développée couvre l'ensemble du cycle d'une application web moderne : modélisation du système, développement backend et frontend, sécurisation par JWT, et déploiement sur une infrastructure cloud distribuée (AWS EC2, Vercel, Neon DB).

CAHIER DES CHARGES ET SPÉCIFICATIONS FONCTIONNELLES

1. Acteurs du Système et Droits d'Accès

L'analyse fonctionnelle de la plateforme identifie trois catégories d'acteurs principaux qui interagissent avec le système. Chacun possède des droits d'accès et des fonctionnalités spécifiques qui garantissent la sécurité et la cohérence de l'utilisation :

- **Visiteur / Acheteur non-authentifié** : Cet acteur représente le point d'entrée sur l'application. Sans avoir besoin de s'authentifier, il a le droit de lire le catalogue global. Il peut effectuer des recherches par mots-clés dans le moteur de recherche interne, filtrer le catalogue par catégories prédéfinies, et ouvrir la page de détails d'une annonce. C'est sur cette page de détails qu'il peut cliquer sur le bouton vert pour contacter directement le vendeur par WhatsApp (avec pré-remplissage du texte du message) ou utiliser l'icône de partage pour envoyer le lien de la publication à un ami ou un collègue.
- **Acheteur authentifié** : Cet acteur a créé un compte utilisateur sur la plateforme et s'est authentifié à l'aide de ses identifiants. En plus des droits conférés au visiteur anonyme, l'acheteur authentifié peut mettre en favoris les annonces qui l'intéressent. Un bouton en forme de cœur sur chaque carte de produit lui permet d'ajouter ou de retirer un produit de ses favoris. Il dispose d'une page dédiée (Favorites.jsx) qui filtre et affiche en temps réel la liste de tous ses produits favoris, lui permettant de suivre de près l'évolution des offres.
- **Vendeur professionnel** : Le rôle de vendeur est accessible à tout utilisateur authentifié qui souhaite proposer des produits ou matériels sur la plateforme. Il accède à un espace de gestion appelé Dashboard (Dashboard.jsx). De là, il peut soumettre de nouveaux produits via un formulaire multipart (titre, marque, prix, description textuelle, photo du produit et numéro WhatsApp spécifique pour l'annonce). Le vendeur est également le seul habilité à éditer les informations de ses propres produits, à modifier la photo, à supprimer l'annonce si le matériel n'est plus disponible, ou à modifier son statut pour le marquer comme 'Vendu'.

2. Spécifications Fonctionnelles Détaillées par Cas d'Utilisation

Les cas d'utilisation modélisent le comportement attendu du système en réponse aux actions des acteurs. Voici les spécifications comportementales pour chaque cas clé du cahier des charges :

- **Enregistrement & Authentification** : L'utilisateur doit pouvoir soumettre son nom, son email, son mot de passe et son numéro de téléphone WhatsApp. Le mot de passe doit être chiffré à l'aide d'un algorithme robuste côté serveur avant l'écriture en base de données. En retour, un jeton d'accès sécurisé JWT est généré et stocké dans le stockage local du navigateur client (localStorage).
- **Publication d'une annonce** : Le vendeur remplit les détails de l'annonce et téléverse une image au format standard (JPEG/PNG). Le système valide que l'utilisateur est authentifié grâce au JWT avant de stocker l'image et les métadonnées dans la base de données. Le produit

nouvellement créé prend par défaut le statut 'Disponible'.

- Prise de contact : L'acheteur clique sur 'Contacter via WhatsApp'. L'application détecte si le numéro WhatsApp du produit est fourni. Elle génère une adresse URL universelle WhatsApp intégrant le texte du message et redirige l'acheteur. Ce processus assure une communication fluide et directe sans intermédiaire de paiement.

- Partage d'annonces : L'utilisateur clique sur le bouton de partage. Si l'API navigator.share est supportée, l'appareil affiche son volet de partage système pour transmettre le lien de l'annonce. Sinon, le lien est copié dans le presse-papiers, permettant un partage manuel instantané.

3. Diagramme de Cas d'Utilisation Global du Système

Le diagramme ci-dessous schématise l'ensemble des interactions entre les différents types d'utilisateurs (Acheteur, Visiteur et Vendeur) et le système central de la marketplace B2B. Ce diagramme structure graphiquement les frontières de l'application et les relations d'inclusion/extension entre cas d'utilisation :

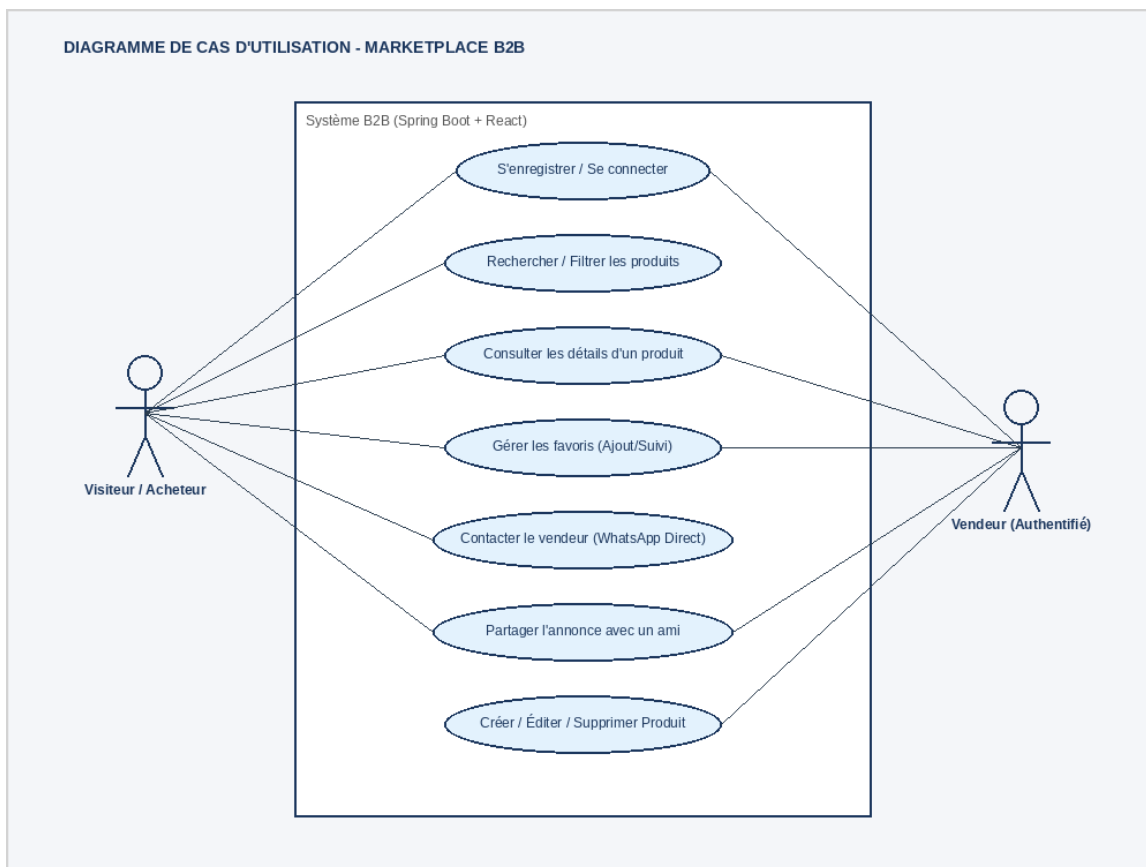


Figure 2.1 : Diagramme de Cas d'Utilisation Global du Système B2B

ARCHITECTURE SYSTÈME ET CHOIX TECHNOLOGIQUES

1. Architecture Globale et Modèle Multi-tiers

L'application repose sur un modèle d'architecture découplé de type multi-tiers (3-Tier Architecture), assurant une séparation stricte des responsabilités et une scalabilité optimale. Ce modèle comprend :

- Le Tier de Présentation (Frontend) : Construit sous forme de Single Page Application (SPA) en React. Ce client léger interagit de manière asynchrone avec le backend via des requêtes HTTP REST. Le fait d'utiliser une SPA garantit une expérience utilisateur fluide, sans rafraîchissement complet des pages à chaque action.
- Le Tier Logique (Backend) : Un serveur Spring Boot exposé sous forme d'API RESTful. Il traite la logique métier, gère l'authentification, valide les entrées et filtre les accès sécurisés aux ressources privées.
- Le Tier de Persistance (Database) : Une base de données relationnelle PostgreSQL hébergée de manière managée sur la plateforme de cloud Neon DB, assurant des transactions ACID et une scalabilité à la demande.

2. Justification du Choix du Backend : Spring Boot (Java 17)

Pour le développement du backend, le framework Spring Boot (Java 17) a été retenu pour ses performances en environnement d'entreprise, sa robustesse et ses capacités d'intégration de sécurité de haut niveau. Les modules spécifiques utilisés incluent :

- Spring Boot Web : Pour exposer l'API RESTful rapidement avec un serveur Tomcat embarqué.
 - Spring Data JPA : Simplifie l'interaction avec le SGBD PostgreSQL grâce au mapping objet-relationnel (ORM Hibernate). Il permet de réduire le code chaudière lié aux transactions et requêtes SQL manuelles.
 - Spring Security : Fournit un framework d'authentification et de contrôle d'accès hautement personnalisable. Dans notre architecture, nous avons surchargé la configuration par défaut pour mettre en place une authentification stateless basée sur des jetons JWT, protégeant ainsi efficacement les routes d'écriture de l'API (création, édition et suppression de produits).
- L'utilisation de Java 17 apporte également de meilleures performances d'exécution, une gestion optimisée de la mémoire et la syntaxe moderne des Records pour les DTOs.

3. Justification du Choix du Frontend : React (Vite, Tailwind)

Le client web a été conçu avec la bibliothèque React et l'outil de build ultra-rapide Vite. Les arguments en faveur de cette stack sont :

- Performance au Build et au Dev : Vite utilise les ES Modules natifs du navigateur pour offrir un rechargement à chaud (HMR) quasi-instantané lors du développement, évitant les lourdeurs de Webpack.
- Rendu Composant React : Permet de concevoir des composants modulaires et réutilisables

(ProductCard, Navbar, FavoriteButton) favorisant la maintenabilité du code.

- Gestion Globale de l'État : Mise en œuvre du React Context API (via MarketplaceContext) pour stocker les informations de session utilisateur (jeton JWT, informations de profil) et la liste partagée des produits, simplifiant les flux de données inter-composants.
- Rapidité de chargement : L'intégration de Tailwind CSS permet de styliser les pages rapidement tout en générant un bundle CSS final minime. Tailwind CSS permet de concevoir une interface responsive de manière déclarative à l'aide de classes utilitaires performantes.

4. Justification de la Base de Données : PostgreSQL Serverless (Neon DB)

PostgreSQL a été sélectionné pour sa conformité totale aux normes ACID, garantissant l'intégrité absolue des transactions (comme la relation d'achat, les associations de favoris, ou l'unicité des comptes d'utilisateurs). Le choix de Neon comme fournisseur de base de données managée apporte plusieurs avantages concurrentiels :

- Architecture Serverless : Neon sépare le stockage du calcul, permettant à la base de données de s'éteindre automatiquement en période d'inactivité et de redémarrer en moins de 2 secondes lors d'un appel API. Cela réduit considérablement les coûts d'infrastructure pendant les heures de faible activité.
- Base de données Cloud-Native : Connexions sécurisées intégrées et console d'administration interactive permettant d'analyser les requêtes lentes et les schémas directement depuis un navigateur web.
- Branchement instantané : Permet de cloner la base de données de production en un clic pour tester des migrations ou des nouveautés sans risque.

5. Containerisation et Infrastructure DevOps de Développement

Pour garantir la portabilité de l'application entre les environnements de développement, de test et de production, une stratégie de conteneurisation intégrale avec Docker a été mise en œuvre. Le backend est compilé dans un conteneur standard reposant sur une image JDK 17 alpine ultra-légère. L'utilisation de Docker Compose simplifie le lancement local de l'application en orchestrant les conteneurs nécessaires avec des configurations standardisées pour les variables d'environnement et l'exposition des ports.

CONCEPTION DU SYSTÈME & MODÉLISATION UML

1. Modélisation Conceptuelle de Données (Diagramme de Classes Métier)

Le diagramme de classes métier (ou modèle de domaine) modélise les concepts clés du problème B2B, sans faire référence aux détails d'implémentation technologiques. Il met en évidence trois concepts clés :

- User : Représente tout professionnel enregistré sur la plateforme. Il possède un identifiant unique, un nom, un e-mail (qui fait office d'identifiant unique de connexion) et un numéro de téléphone WhatsApp pour les prises de contact.
- Produit : Représente une annonce de produit/matériel publiée sur la plateforme. Chaque produit possède un prix, un statut de vente (Disponible ou Vendu), une image, une date de publication, une marque et un compteur de vues.
- CategorieProduit : Sert à classifier logiquement les produits pour faciliter la navigation interentreprises.

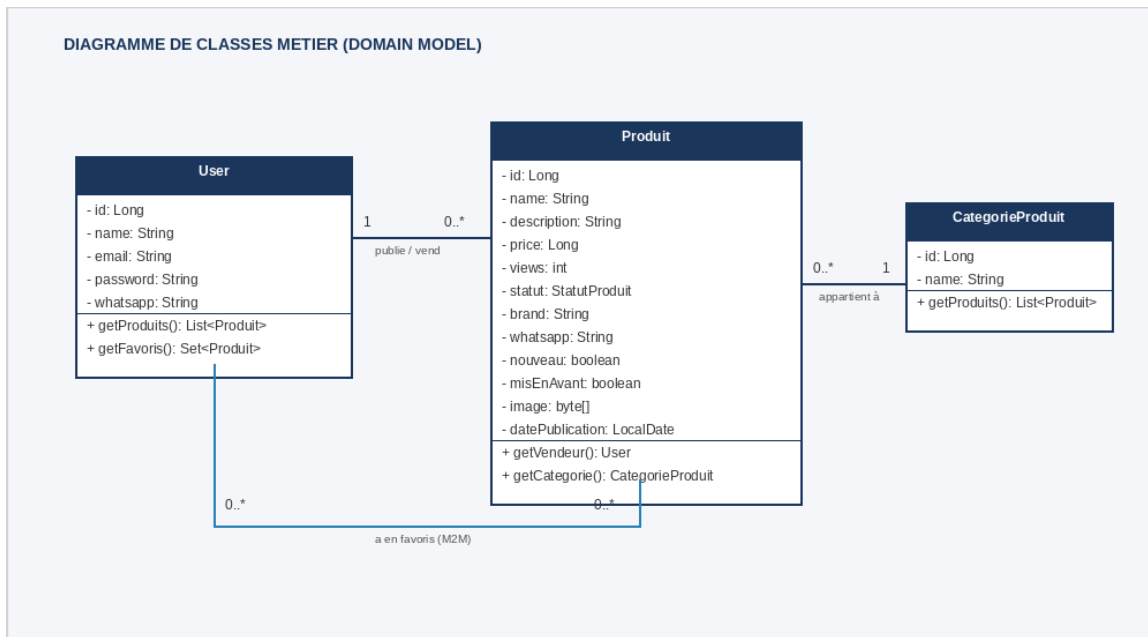


Figure 4.1 : Diagramme de Classes Métier (Domain Model)

2. Spécification Technique des Entités (Diagramme de Classes Technique)

Le diagramme de classes technique détaille les relations structurales au niveau de l'implémentation Java. Il intègre les différentes couches de l'architecture logicielle : les Contrôleurs REST qui interceptent les requêtes HTTP, les Services qui orchestrent la logique

métier, les Repositories (Spring Data) responsables des requêtes de persistance, et les Entités JPA mappées à notre base de données PostgreSQL Neon :

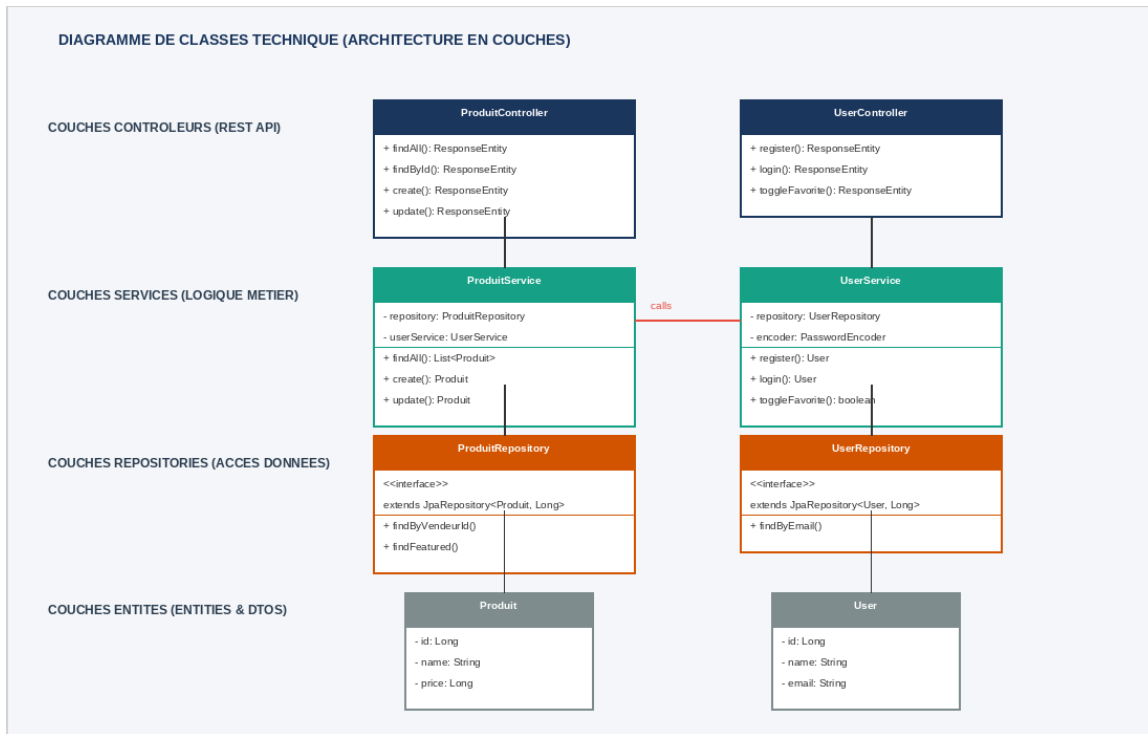


Figure 4.2 : Diagramme de Classes Technique (Architecture en Couches Backend)

3. Dictionnaire de Données et Structure des Tables SQL

Les relations UML sont transposées au niveau de la base de données relationnelle sous forme de tables SQL structurées de la façon suivante :

- Table `users` : Contient les informations des utilisateurs. Un index unique est appliqué sur l'e-mail pour garantir son unicité physique.
- Table `categories\_produit` : Contient le catalogue de catégorisation.
- Table `produits` : Contient le catalogue d'annonces. Une clé étrangère `vendeur\_id` pointe vers la table `users` et une clé `categorie\_id` pointe vers la table `categories\_produit`.
- Table d'association `user\_favoris` : Table de jointure modélisant la relation many-to-many entre `users` (colonne `user\_id`) et `produits` (colonne `produit\_id`).

L'utilisation d'index sur les clés étrangères (`vendeur\_id`, `categorie\_id`) permet d'optimiser les performances des requêtes de jointure lorsque le volume d'annonces augmente.

MODÈLE DE SÉCURITÉ ET AUTHENTIFICATION JWT

1. Principes Fondamentaux de la Sécurité Stateless avec JWT

Pour assurer une architecture RESTful évolutive et indépendante (stateless), la plateforme utilise des jetons JWT (JSON Web Tokens) pour sécuriser ses échanges. L'utilisation du JWT évite l'utilisation de sessions HTTP côté serveur qui consommeraient trop de ressources en mémoire vive. Un jeton JWT est composé de trois parties distinctes séparées par des points : le Header (indiquant l'algorithme de hachage), le Payload (contenant les revendications ou claims comme l'ID utilisateur et la date d'expiration) et la Signature (calculée à l'aide d'une clé secrète serveur pour interdire toute falsification). Le client stocke le jeton et le transmet dans l'en-tête `Authorization` à chaque requête de modification de données.

2. Implémentation du Filtre de Sécurité Customisé Spring Boot

Dans le backend Spring Boot, la sécurité est mise en place par le composant `SecurityConfig` et un filtre de servlet personnalisé nommé `JwtAuthFilter` (qui hérite de `OncePerRequestFilter`). Le flux de sécurité se déroule ainsi :

1. L'application cliente (React) envoie une requête HTTP vers une route protégée, en incluant l'en-tête standard : `Authorization: Bearer <token\_jwt>`.
2. Le filtre `JwtAuthFilter` intercepte la requête HTTP avant qu'elle n'atteigne le contrôleur Spring Web.
3. Le filtre extrait le préfixe 'Bearer ' et analyse le jeton à l'aide du composant utilitaire `JwtUtil`.
4. Si le jeton est valide, l'ID utilisateur est extrait et injecté sous forme d'objet d'authentification principal dans le contexte de sécurité de Spring Security (`SecurityContextHolder`).
5. Le contrôleur REST correspondant peut alors récupérer directement l'identifiant de l'utilisateur connecté via l'annotation `@AuthenticationPrincipal Long userId`.
6. Si le jeton est manquant, corrompu ou expiré, la requête est rejetée avec un code HTTP 401 Unauthorized.

3. Gestion de la Politique de Partage des Ressources (CORS)

Puisque le frontend React et le backend Spring Boot sont hébergés sur des serveurs et des domaines distincts (Vercel et AWS EC2), le navigateur applique la politique de sécurité de même origine (Same-Origin Policy). Pour permettre les requêtes cross-origin, le composant `CorsConfig` du backend Spring Boot configure explicitement les règles autorisées :

- Origines autorisées : Les URL spécifiques du frontend hébergé sur Vercel (ainsi que `http://localhost:5173` pour le développement local).
- Méthodes HTTP autorisées : GET, POST, PUT, PATCH, DELETE et OPTIONS.
- En-têtes autorisés : Content-Type, Authorization, et tous les en-têtes personnalisés.
- Credentials : Les cookies ou en-têtes d'authentification sont autorisés à transiter en toute sécurité.

CONCEPTION DÉTAILLÉE DES FLUX ET SÉQUENCES

1. Diagramme de Séquence : Publication d'un Produit par le Vendeur

Ce diagramme de séquence montre l'ensemble des échanges réseau et des opérations logiques internes requis lorsqu'un vendeur connecté décide de publier une nouvelle annonce de produit sur la plateforme. Ce flux prend en charge le traitement d'une requête multipart pour le téléversement de la photo du produit :

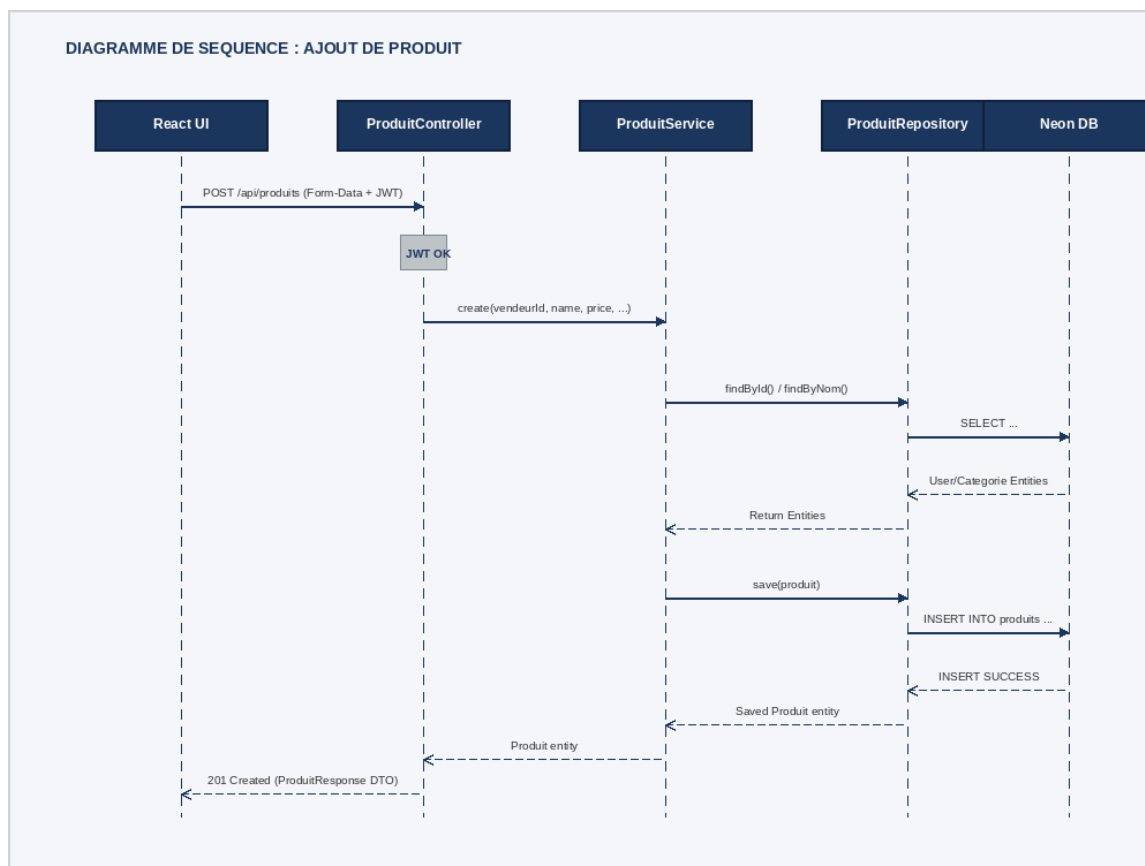


Figure 6.1 : Diagramme de Séquence - Publication d'un Produit

2. Diagramme de Séquence : Prise de Contact Directe via WhatsApp

La mise en relation directe est une fonctionnalité essentielle de notre marketplace B2B. Le diagramme suivant représente le processus d'interaction où un acheteur (authenticé ou non) consulte la fiche d'un produit, ce qui incrémente le compteur de vues de l'annonce côté backend, puis clique sur le bouton de contact WhatsApp pour entamer instantanément une conversation de négociation :

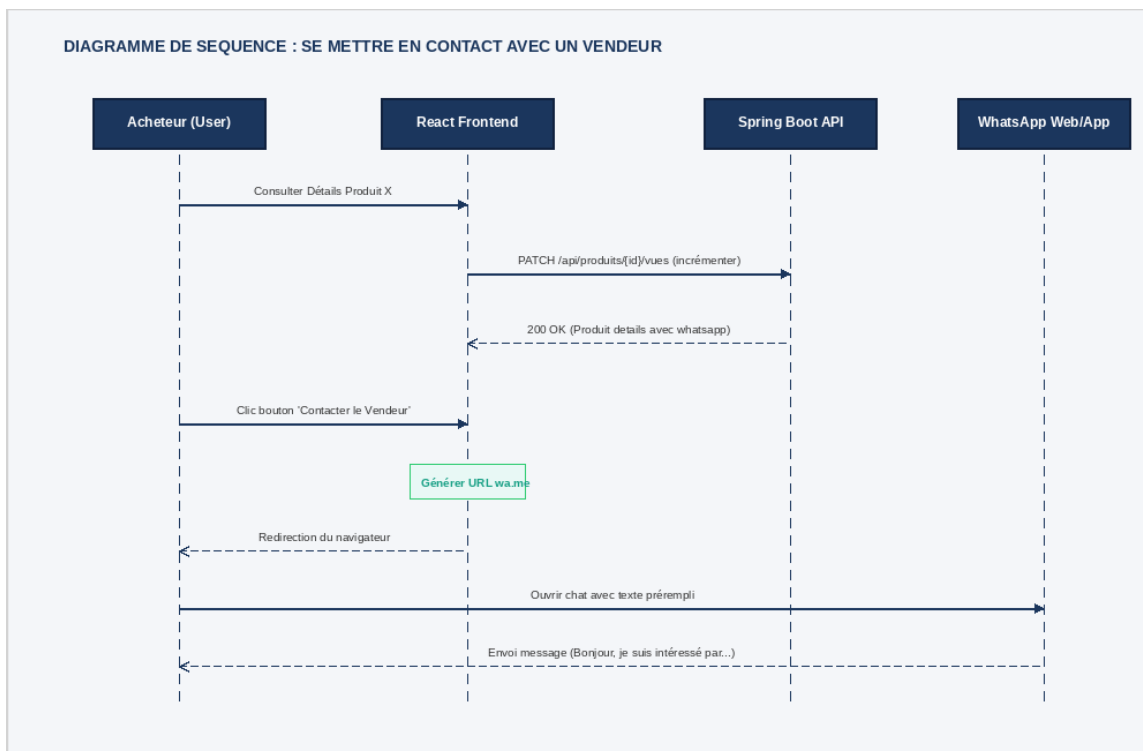


Figure 6.2 : Diagramme de Séquence - Contact Vendeur via WhatsApp

3. Diagramme de Séquence : Partage de l'Annonce avec un Ami

Pour favoriser le partage viral des opportunités d'affaires (B2B), nous avons spécifié un mécanisme de partage intégré. Ce diagramme décrit le cas d'utilisation où un professionnel partage une annonce active avec un collègue ou un ami via les fonctionnalités de son appareil en s'appuyant sur l'API Web Share native :

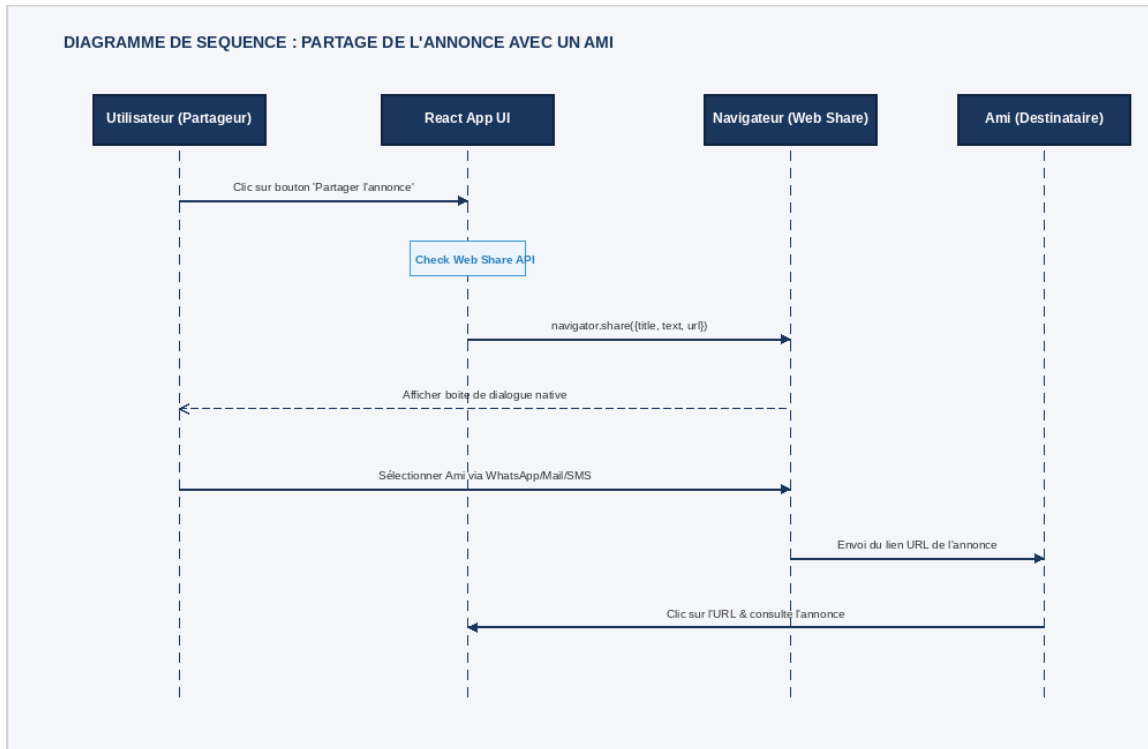


Figure 6.3 : Diagramme de Séquence - Partage d'une Publication

4. Implémentation du Partage Frontend (Web Share API et Presse-Papiers)

Pour assurer le partage d'annonces de manière professionnelle et fluide, le code frontend React déploie une fonction de partage dynamique. Si le navigateur prend en charge l'API Web Share de type `navigator.share()`, l'application propose l'ouverture du volet de partage système de l'OS (permettant un partage vers des applications comme WhatsApp, Slack, Teams, Gmail, ou par SMS). En guise de repli (fallback) pour les navigateurs non compatibles, le système copie instantanément le lien court de l'annonce dans le presse-papiers de l'utilisateur tout en affichant un toast de notification élégant indiquant 'Lien copié dans le presse-papiers !'. Voici le principe de cette implémentation :

```

[CODE JS: LOGIQUE DE PARTAGE D'ANNONCE] const handleShare = async () => {
  const shareData = {
    title: product.name,
    text: `Consultez ce produit sur notre marketplace B2B : ${product.name}`,
    url: window.location.href
  };
  if (navigator.share) {
    try {
      await navigator.share(shareData);
    } catch (err) {
      console.log('Erreur lors du partage natif:', err);
    }
  }
}
  
```

```
    }  
  } else {  
    navigator.clipboard.writeText(window.location.href);  
    showToast('Lien de l\'annonce copié !');  
  }  
};
```


DEVOPS, DOCKER & WORKFLOW DE DÉPLOIEMENT

1. Configuration des Conteneurs via Dockerfile Multi-stage

Dans une démarche moderne d'automatisation du déploiement, nous utilisons un workflow basé sur Docker. Le backend Spring Boot possède un fichier `Dockerfile` multi-stage permettant de compiler puis de packager l'application de manière optimale, en limitant la taille de l'image de production finale. La compilation est effectuée dans un premier environnement Maven temporaire, et seul le jar final est copié dans l'environnement d'exécution.

```
[DOCKERFILE BACKEND (Spring Boot)] # Stage 1: Build the Maven application
FROM maven:3.8.5-openjdk-17 AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Create runtime image
FROM openjdk:17-jdk-alpine
WORKDIR /app
COPY --from=builder /app/target/b2b-0.0.1-SNAPSHOT.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

2. Orchestration Locale Multiconteneur avec Docker Compose

Pour exécuter l'infrastructure complète localement lors des phases de développement ou de test, un fichier `compose.yaml` est mis en place dans le répertoire backend. Ce fichier orchestre une base de données PostgreSQL locale ainsi que l'application Spring Boot conteneurisée. En production, la base PostgreSQL locale est substituée par la base PostgreSQL serverless Neon DB pour de meilleures garanties de résilience, de sauvegardes automatiques et de performance. Cela garantit que l'équipe de développement travaille dans des conditions proches de la production.

ARCHITECTURE DE DÉPLOIEMENT CLOUD

1. Vue d'Ensemble de l'Architecture Déployée

L'application est déployée sur trois services cloud distincts, chacun hébergeant une couche spécifique de l'architecture :

- Le backend Spring Boot tourne sur une instance AWS EC2 (machine virtuelle Linux dans le cloud d'Amazon).
- Le frontend React est hébergé sur Vercel, plateforme spécialisée dans l'hébergement d'applications web statiques.
- La base de données PostgreSQL est hébergée sur Neon, un service de base de données cloud managé.

Ces trois composants communiquent entre eux via le protocole HTTPS. Le frontend envoie des requêtes HTTP vers l'API REST exposée par le backend EC2. Le backend, quant à lui, se connecte à la base de données Neon via une URL de connexion sécurisée (SSL/TLS).

[FLUX DE COMMUNICATION ENTRE COMPOSANTS] Frontend (Vercel) —HTTPS—> API REST (AWS EC2 / Spring Boot) —JDBC/SSL—> Base de données (Neon / PostgreSQL)

2. Hébergement du Backend sur AWS EC2

Le backend Spring Boot est déployé sous forme de conteneur Docker sur une instance Linux AWS EC2. Un serveur web Nginx fait office de point d'entrée : il reçoit les requêtes HTTPS arrivant sur le domaine public et les redirige vers l'application Spring Boot en interne. Les groupes de sécurité AWS restreignent l'accès réseau : seul le trafic HTTP/HTTPS public est autorisé, le port interne de Spring Boot n'est pas exposé directement sur internet.

3. Hébergement du Frontend sur Vercel

Le client React est compilé (`npm run build`) en un ensemble de fichiers statiques (HTML, JS, CSS) puis déployé sur Vercel via une intégration GitHub. À chaque mise à jour du code sur le dépôt, Vercel relance automatiquement un build et publie la nouvelle version.

4. Communication entre les Composants

Le frontend communique avec le backend via des requêtes HTTP REST. L'URL de base de l'API (`VITE\_API\_URL`) est configurée comme variable d'environnement dans Vercel et pointe vers le domaine public de l'instance EC2. Toutes les communications transitent en HTTPS pour garantir la confidentialité des données (en particulier les jetons JWT et les informations utilisateurs).

La base de données Neon est accessible par le backend uniquement, via une URL de connexion JDBC sécurisée. Le frontend n'a jamais de connexion directe à la base de données :

toutes les opérations passent obligatoirement par l'API REST du backend, qui valide les droits d'accès avant chaque action.

GUIDE D'UTILISATION COMPLET DE LA PLATEFORME B2B

1. Inscription, Connexion et Gestion du Profil Utilisateur

Pour tirer pleinement parti des fonctionnalités de la plateforme B2B, l'utilisateur doit disposer d'un compte actif. La procédure s'articule autour des étapes suivantes :

- Inscription : L'utilisateur se rend sur la page d'inscription. Il saisit son nom de société ou d'individu, son adresse e-mail professionnelle unique, son mot de passe (qui doit comporter au moins 6 caractères pour des raisons de sécurité) et son numéro de téléphone WhatsApp au format international (ex: +22890000000). Le clic sur le bouton d'inscription soumet les données et connecte automatiquement l'utilisateur en générant un jeton de session JWT.
- Connexion : Si l'utilisateur possède déjà un compte, il renseigne son adresse e-mail et son mot de passe sur la page de connexion.
- Suivi de Session : L'application conserve les informations de l'utilisateur connecté en haut à droite de l'écran dans un menu de profil personnalisé. Un bouton de déconnexion permet de détruire le jeton stocké dans le navigateur pour sécuriser son poste de travail.

2. Recherche Avancée, Filtres par Catégorie et Liste de Favoris

La page d'accueil (Accueil.jsx) est conçue pour être claire et intuitive. Elle présente un en-tête (Hero section) invitant les professionnels à explorer les offres industrielles et matérielles actives.

- Liste des Produits : Les annonces s'affichent sous forme de grille de cartes produit. Chaque carte indique si le produit est 'Nouveau', son prix, sa catégorie et un aperçu de son image.
- Filtres Rapides : Des boutons de filtre par catégorie permettent en un clic d'isoler les matériels informatiques ou les fournitures de bureau.
- Barre de Recherche : L'utilisateur peut saisir des mots-clés dans le champ de recherche pour filtrer dynamiquement les annonces.
- Liste des Favoris : Les acheteurs connectés peuvent cliquer sur le cœur d'une carte produit pour l'ajouter aux favoris. La liste est accessible dans l'onglet 'Favoris' (Favorites.jsx).

3. Espace Vendeur : Ajout, Édition et Clôture d'une Annonce

Tout utilisateur authentifié a accès au rôle de Vendeur et dispose de son propre Tableau de Bord (Dashboard.jsx) :

- Publier une annonce : Le vendeur clique sur 'Ajouter un produit'. Un formulaire complet lui permet de renseigner le nom, la description technique, le prix, la marque, le numéro WhatsApp et de téléverser l'image du produit. À la soumission, le produit est publié immédiatement.
- Modifier une annonce : Le vendeur peut corriger des erreurs de prix, de description ou téléverser une nouvelle photo à tout moment.
- Clôturer/Marquer comme Vendu : Le vendeur peut basculer le statut du produit de 'Disponible' à 'Vendu'. Un badge rouge 'VENDU' s'affichera alors sur l'annonce, et le bouton WhatsApp sera automatiquement désactivé pour éviter de nouvelles sollicitations.

4. Utilisation Pratique des Boutons de Contact et de Partage d'Annonces

La fonctionnalité de partage d'annonce a été spécifiquement conçue pour encourager la diffusion collaborative des offres :

1. Accédez à la page de détails de l'annonce que vous souhaitez partager.
2. Repérez le bouton 'Partager' situé à côté des informations de contact.
3. Clic sur le bouton : si vous naviguez depuis un smartphone, un menu système s'ouvre pour envoyer l'annonce directement à un ami sur WhatsApp, Messenger, par e-mail ou Slack.
4. Si le partage natif n'est pas disponible (sur PC), l'application copie automatiquement l'URL directe de l'annonce dans le presse-papiers, et vous pouvez la coller où vous le souhaitez. De plus, le bouton WhatsApp permet d'entamer une conversation directe avec le vendeur avec un texte prérempli.

ADMINISTRATION ET MAINTENANCE DE LA PLATEFORME

1. Gestion de la Base de Données via la Console Neon

La base de données PostgreSQL hébergée sur Neon est administrable directement via sa console web. Cette interface permet d'exécuter des requêtes SQL pour vérifier les données stockées (liste des utilisateurs, état des annonces, associations de favoris), de consulter les tables et leur schéma, et de modifier des enregistrements si nécessaire. Neon propose également un système de branches de base de données, permettant de tester des modifications de schéma sur une copie isolée sans affecter les données de production.

2. Supervision de l'Application Backend

Le backend Spring Boot s'exécutant dans un conteneur Docker sur l'instance EC2, les journaux applicatifs (logs) sont accessibles via la commande Docker standard (`docker logs`). Ces logs permettent de diagnostiquer les erreurs éventuelles : échecs d'authentification JWT, problèmes de connexion à la base de données, ou requêtes REST malformées.

Les logs Nginx enregistrent toutes les requêtes HTTP entrantes (méthode, URL, code de réponse), ce qui permet de vérifier que le routage entre le frontend Vercel et le backend EC2 fonctionne correctement.

VALIDATION ET TESTS DE L'APPLICATION

1. Tests Fonctionnels et Vérification des Scénarios Clés

La phase de validation de notre projet a consisté à vérifier manuellement et de façon systématique le bon déroulement de chacun des scénarios définis dans le cahier des charges. Pour chaque cas d'utilisation principal, nous avons élaboré un protocole de test qui spécifie les données d'entrée, les actions à effectuer et le résultat attendu.

Voici les principaux scénarios validés :

- Inscription / Connexion : Création d'un compte avec des données valides puis avec un e-mail déjà utilisé pour vérifier la gestion d'erreur.
- Publication d'annonce : Soumission du formulaire complet avec téléversement d'image. Vérification de l'apparition immédiate du produit dans le catalogue.
- Sécurité JWT : Tentative d'appel direct à l'API REST de création de produit sans jeton d'authentification — vérification du retour HTTP 401 Unauthorized.
- Contact WhatsApp : Clic sur le bouton 'Contacter le Vendeur' depuis la fiche produit — vérification que l'application WhatsApp s'ouvre avec le bon numéro et le texte prérempli.
- Partage d'annonce : Test du bouton de partage sur mobile (Android) — vérification de l'affichage du volet natif du système. Test sur navigateur de bureau — vérification que l'URL est bien copiée dans le presse-papiers.
- Gestion des favoris : Ajout et retrait d'un produit depuis la liste de favoris, avec vérification de la mise à jour visuelle immédiate.

2. Résultats et Points d'Amélioration

L'ensemble des fonctionnalités du cahier des charges a été implémenté et validé. Quelques points méritent cependant d'être améliorés dans une version ultérieure :

- La gestion CORS entre Vercel et EC2 nécessite une configuration explicite des origines autorisées dans la classe `CorsConfig` de Spring Boot pour que le frontend puisse appeler l'API sans erreurs.
- Le téléversement d'images (stockées en `byte[]` dans PostgreSQL) requiert une gestion soigneuse du format `multipart/form-data` côté backend et de l'objet `FormData` côté React.
- La compatibilité de l'API Web Share : `navigator.share()` n'est pas disponible sur tous les navigateurs. Un mécanisme de repli (`fallback`) copie l'URL dans le presse-papiers lorsque le partage natif est indisponible.

[RÉSULTAT DE LA VALIDATION] L'ensemble des fonctionnalités du cahier des charges initial a été implémenté et validé avec succès. Les tests ont été conduits sur Google Chrome, Firefox et sur mobile Android. Aucun défaut bloquant n'a été identifié lors de la validation finale.

CONCLUSION ET PERSPECTIVES

La plateforme marketplace B2B réalisée dans le cadre de ce cours de Conception des Systèmes d'Information constitue une application web fullstack fonctionnelle et déployée. Elle couvre l'ensemble des besoins définis dans le cahier des charges : publication d'annonces, recherche et filtrage, gestion des favoris, contact direct via WhatsApp et partage d'annonces. L'architecture adoptée — Spring Boot, React, PostgreSQL Neon, Docker et déploiement cloud — reflète les bonnes pratiques actuelles du développement logiciel.

Bilan Technique du Projet

Sur le plan technique, le projet a permis de mettre en œuvre les concepts suivants :

- Architecture multicouche (Controller, Service, Repository, Entity) avec Spring Boot.
- Développement d'une Single Page Application en React avec gestion de l'état global via Context API.
- Authentification stateless par JSON Web Tokens (JWT) avec filtre de sécurité Spring Security.
- Modélisation UML complète : diagramme de cas d'utilisation, diagramme de classes métier et technique, diagrammes de séquences pour les flux principaux.
- Containerisation avec Docker et déploiement distribué sur AWS EC2, Vercel et Neon DB.
- Intégration de la Web Share API et du contact WhatsApp pour une expérience utilisateur moderne.

Perspectives d'Amélioration

Des évolutions peuvent être envisagées pour enrichir la plateforme :

- Un système de messagerie interne en temps réel (WebSocket) pour permettre la négociation directement sur la plateforme.
- Un moteur de recommandation basé sur l'historique de navigation et les favoris.
- Un backoffice d'administration permettant de modérer les annonces et gérer les utilisateurs.
- Des notifications push pour alerter un vendeur lors d'une prise de contact.

La plateforme développée présente un potentiel réel et pourrait, avec ces améliorations progressives, constituer une solution utilisable dans un contexte professionnel.